ResIR Compiler Project: Technical Implementation Report

Compiler Theory & Practice (CPS2000)

Kurt Abela (150106L) | kurt.abela.24@um.edu.mt

1. Structure of Lexer and Parser.....3
1.1 Lexical Analyzer (Lexer)..... 3
1.2 Parser.....3
2. Grammar Deviations.....5
3. Semantic Pass Invariants.....6
4. Code Examples and Generated C..... 7
Example 1: Absolute Difference..... 7
Example 2: Factorial (Loop)..... 8
Example 3: Member Access (Structs)..... 9
5. Datalog Static Analysis.....11
5.1 Fact Extraction..... 11
5.2 Datalog Rules (analysis.dl).....11
5.3 Derived Results..... 11
6. Future Improvements..... 13
7. Quality Assurance..... 14
8. Short Video Presentation.....19
9. Plagiarism Form..... 20

This report documents the architectural design, implementation details, and static analysis capabilities of the **ResIR Compiler**, a C++-based toolchain that translates Intermediate Representation (IR) code into target C source code while performing liveness analysis via Datalog.

1. Structure of Lexer and Parser

The front-end is designed for high performance and predictability, utilizing a two-stage process to transform raw text into a Control Flow Graph (CFG).

1.1 Lexical Analyzer (Lexer)

The lexer is implemented as a **table-driven Deterministic Finite Automaton (DFA)**.

- **Mechanism**: It classifies characters into `InputClass` categories (e.g., `DIGIT`, `ALPHA`, `OPERATOR`) and uses a 2D transition table to determine the next state.
- **Key Logic**:

```
// From lexer.cpp
State current = State::START;
while (current != State::DONE && current != State::ERROR) {
    char c = peek();
    InputClass ic = classify(c);
    current = TRANSITION_TABLE[static_cast<int>(current)][static_cast<int>(ic)];
    // ... advance position ...
}
```

- **Keyword Handling**: After identifying an identifier, the lexer performs an $O(1)$ lookup in a `std::unordered_map` containing 43 reserved keywords (e.g., `cjump`, `member_ptr`, `locals`) to distinguish them from variable names.

1.2 Parser

The parser follows a **recursive-descent** strategy. Since the ResIR grammar is LL(1), the parser requires only a single token of lookahead and no backtracking.

- **Structure**: Each grammar rule corresponds to a private method (e.g., `parse_block()`, `parse_stmt()`).
- **CFG Generation**: Instead of building a generic Abstract Syntax Tree (AST), the parser directly populates a `Program` structure composed of basic blocks, which simplifies subsequent analysis passes.

2. Grammar Deviations

Based on the project guidelines provided in the specification, the following deviations were implemented to enhance the language's utility:

- **Namespaced Identifiers**: The implementation supports `::` in function names (e.g., `Math::abs_diff`), whereas the base grammar expects simple alphanumeric identifiers.
- **Extern Type Declarations**: The parser includes an `extern type` rule to support complex memory layouts, which was not present in the initial minimal ResIR spec.
- **Member Pointer Arithmetic**: The addition of the `member_ptr` instruction allows for type-safe pointer offsets into structs: `%ptr = member_ptr %base_ptr, Path::field_name;`

3. Semantic Pass Invariants

The validator implements a rigorous multi-phase check to ensure the ResIR is sound before it reaches the code generator. While Phases 1–3 verify declarations and labels, **Phase 4 (`validate_all_blocks`)** ensures the semantic integrity of the execution flow:

- **Operand Declaration Check**: For every statement, the validator ensures all source operands exist in the symbol table.
- **Type Narrowing and Compatibility**: The `infer_rhs_type` method strictly enforces that arithmetic operations (e.g., `bin add`) use identical numeric types (no implicit casting).
- **Pointer Dereference Safety**: The `LOAD` opcode is only valid if the operand is of type `ptr<T>`, and the result is strictly inferred as `T`.
- **Terminator Consistency**: For `cjump`, the validator verifies the condition is explicitly a `bool` type and that both the `true` and `false` target labels exist.

4. Code Examples and Generated C

Below are three ResIR input functions and their corresponding C translations.

Example 1: Absolute Difference

ResIR Input:

```
function Math::abs_diff(%a: i32, %b: i32) -> i32 {
  locals { %d: i32; %is_neg: bool; }
  entry bb0;
  bb0:
    %d = bin sub %a, %b;
    %is_neg = bin lt %d, const 0;
    cjump %is_neg, bb1, bb2;
  bb1:
    %d = un neg %d;
    return %d;
  bb2:
    return %d;
}
```

Generated C:

```
int32_t Math_abs_diff(int32_t a, int32_t b) {
    int32_t d;
    bool is_neg;
    goto bb0;
bb0:
    d = a - b;
    is_neg = d < 0;
```

```
    if (is_neg) goto bb1; else goto bb2;
bb1:
    d = -d;
    return d;
bb2:
    return d;
}
```

Example 2: Factorial (Loop)

ResIR Input:

```
function Math::fact(%n: i32) -> i32 {
  locals { %res: i32; %cond: bool; }
  entry start;
  start:
    %res = const 1;
    goto loop;
  loop:
    %cond = bin gt %n, const 1;
    cjump %cond, body, done;
  body:
    %res = bin mul %res, %n;
    %n = bin sub %n, const 1;
    goto loop;
  done:
    return %res;
}
```

Generated C:

```
int32_t Math_fact(int32_t n) {
```

```
    int32_t res;
    bool cond;

    // Jump to the entry block of the function.
    goto start;

    start:
        res = 1;
        goto loop;

    loop:
        cond = n > 1;
        if (cond) goto body; else goto done;

    body:
        res = res * n;
        n = n - 1;
        goto loop;

    done:
        return res;
}
```

Example 3: Member Access (Structs)

ResIR Input:

```
extern type Point { x: f64; y: f64; }
function GetX(%p: ptr<Point>) -> f64 {
  locals { %x_ptr: ptr<f64>; %val: f64; }
  entry bb0;
```

```
  bb0:
    %x_ptr = member_ptr %p, Point::x;
    %val = load %x_ptr;
    return %val;
}
```

Generated C:

```
double GetX(Point* p) {
    double* x_ptr;
    double val;

    goto bb0;

    bb0:
        x_ptr = &p->x;  // Result of member_ptr logic
        val = *x_ptr;   // Result of load logic
        return val;
}
```

5. Datalog Static Analysis

The compiler exports CFG information to Soufflé Datalog to perform **Live Variable Analysis**.

5.1 Fact Extraction

The compiler generates three TSV files:

- `edge.facts: (FromBlock, ToBlock)`
- `def.facts: (Block, Variable)` — variables defined in a block.
- `use.facts: (Block, Variable)` — variables read in a block.

5.2 Datalog Rules (`analysis.dl`)

Liveness is calculated using the following recursive logic:

$$LiveIn(B) = Use(B) \cup (LiveOut(B) \backslash Def(B))$$

$$LiveOut(B) = \bigcup_{S \in succ(B)} LiveIn(S)$$

Implementation:

```
live_out(B, X) :- edge(B, S), live_in(S, X).
live_in(B, X)  :- use(B, X).
live_in(B, X)  :- live_out(B, X), !def(B, X).

// Derived Analysis: Dead Assignment Detection
dead_def(B, X) :- def(B, X), !live_out(B, X).
```

5.3 Derived Results

Running the analysis on `Math::abs_diff` yields:

- `live_out(bb0, r)`: Even though `%r` is defined in `bb1`, it appears live-out of `bb0` because it must be eventually available for the return path. Datalog propagates liveness backwards from the final `return %r` in `bb1` through the `edge(bb0, bb1)` fact.

- `dead_def(bb0, is_neg)`: The variable `%is_neg` is flagged as a dead definition upon exiting `bb0`. This occurs because, after its use in the `cjump` terminator within `bb0`, no successor block (`bb1` or `bb2`) ever reads its value again, satisfying the rule: `dead_def(B, X) :- def(B, X), !live_out(B, X)`.

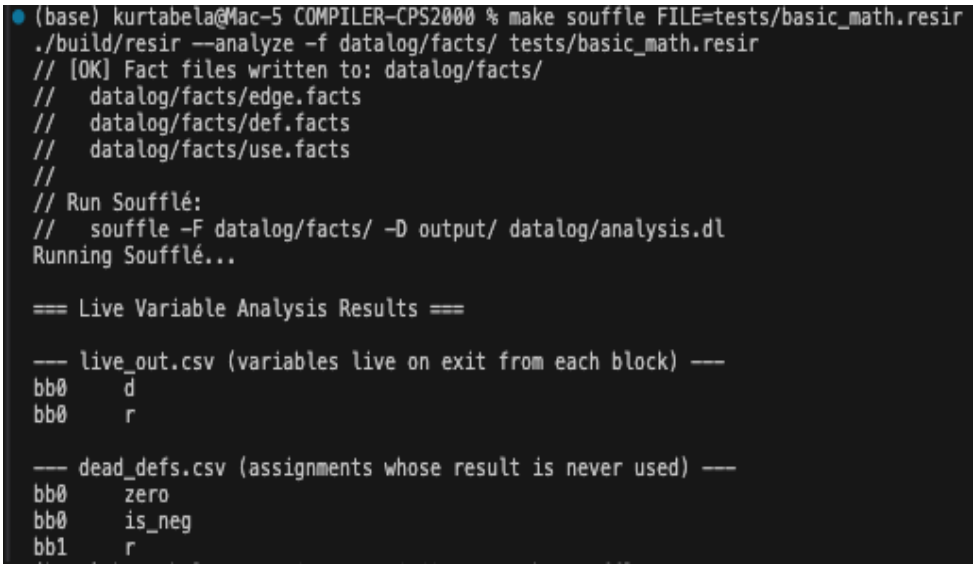


Figure 1 - Running Souffle to Create the Datalog Files for basic_math.resir

6. Future Improvements

- **Constant Folding Pass**: Implement a pre-codegen pass in C++ to evaluate constant expressions (e.g., `bin add const 1, const 2`) at compile-time.
- **Register Allocation Hints**: Use the `live_out` Datalog results to generate C code that hints to the C compiler which variables should be stored in registers using the `register` keyword.
- **E-Graph Integration**: Fully integrate the `egg` library to perform equality saturation on algebraic identities, allowing the compiler to simplify complex math operations before liveness analysis.

7. Quality Assurance

The project includes a robust test suite covering all phases.

```
(base) kurtabela@Mac-5 COMPILER-CPS2000 % make test

--- Lexer tests ---
./build/resir_lexer_test
============================
 ResIR Lexer — Unit Tests
============================

[Keywords]
  PASS  function/locals/entry
  PASS  all statement keywords

[Primitive types]
  PASS  bool i32 i64 u32 f64

[Operators]
  PASS  all operators

[Bool/null literals]
  PASS  true false null

[Identifiers]
  PASS  plain ident text

[Locals]
  PASS  local tokens carry %name

[Labels]
  PASS  labels

[Integer literals]
  PASS  integer values

[Float literals]
  PASS  float values

[Punctuation]
  PASS  all punctuation

[Comments]
  PASS  line comment skipped

[Position tracking]
  PASS  i32 on line 1 col 1
  PASS  %x on line 2 col 3

[Error handling]
  PASS  lone `-` is an error
  PASS  `%123` invalid local
  PASS  lone `@` is an error
  PASS  lone `/` is an error

[Full snippet — abs_diff from spec]
  PASS  abs_diff tokenises without error
  PASS  first token is `function`
  PASS  path Math::abs_diff present

[Full snippet — struct access from spec]
  PASS  struct_access tokenises without error
  PASS  starts with `extern`

============================
 Results: 23 passed, 0 failed
============================
```

Figure 2 - Terminal Output for Task 1; Lexer

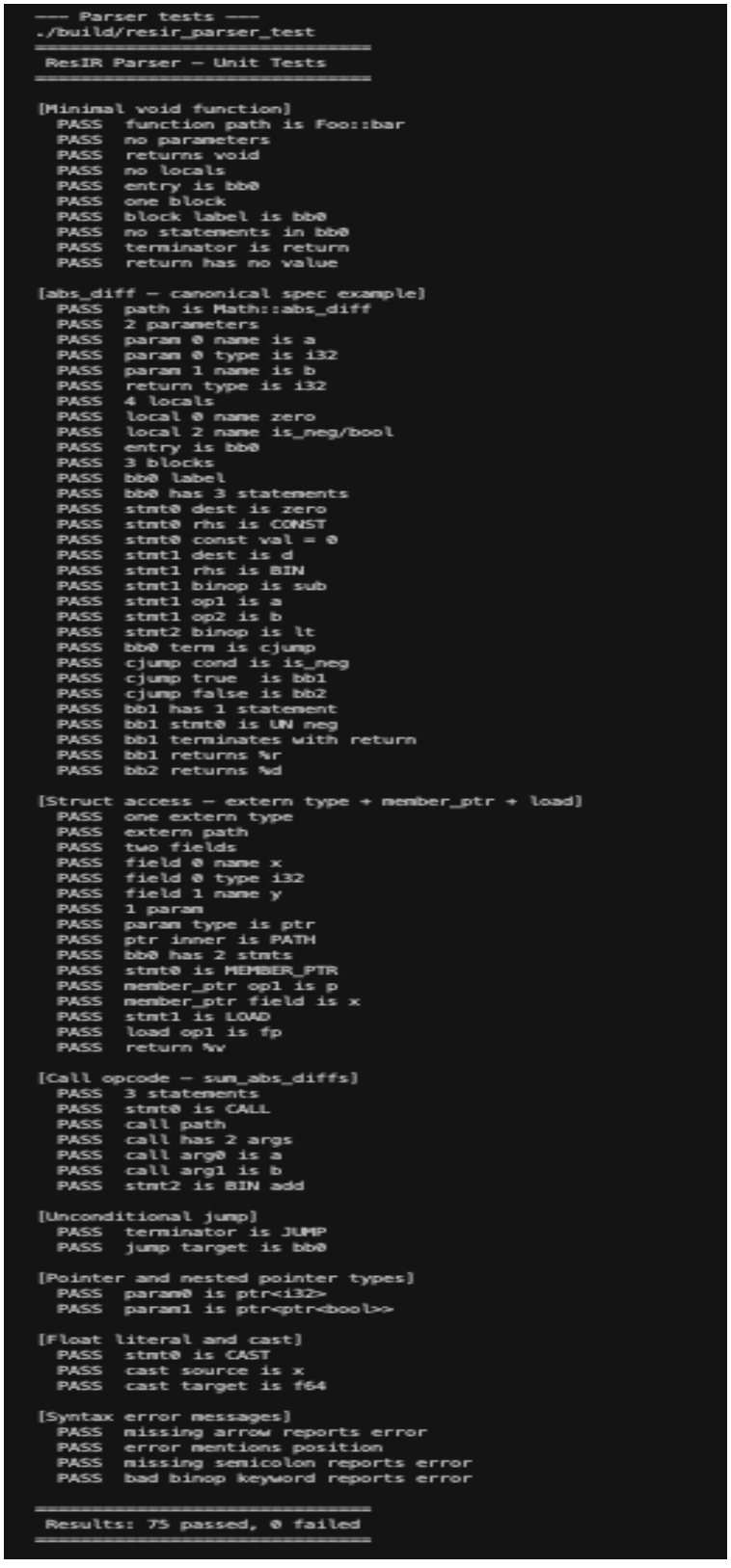


Figure 3 - Terminal Output for Task 2; Parser



Figure 4 - Terminal Output for Task 3; Validator



Figure 5 - Terminal Output for Task 4; C Code Generation



Figure 6 - Terminal Output for Task 5; Datalog Fact Extractor

The compiler demonstrates a successful end-to-end pipeline from Micro-syntax IR to executable C, with verifiable static analysis.

8. Short Video Presentation

Access the short video demonstration of the ResIR project via this link: https://drive.google.com/file/d/1VkpwPgHf0CMCuC7NhO9WXw2bVs6vI9sX/view?usp=drive_link

9. Plagiarism Form

FACULTY OF INFORMATION AND COMMUNICATION TECHNOLOGY

Declaration

Plagiarism is defined as "the unacknowledged use, as one's own work, of work of another person, whether or not such work has been published" (Regulations Governing Conduct at Examinations, 1997, Regulation 1 (viii), University of Malta).

I / We*, the undersigned, declare that the [assignment / Assigned Practical Task report / Final Year Project report] submitted is my / our* work, except where acknowledged and referenced.

I / We* understand that the penalties for making a false declaration may include, but are not limited to, loss of marks; cancellation of examination results; enforced suspension of studies; or expulsion from the degree programme.

Work submitted without this signed declaration will not be corrected, and will be given zero marks.

* Delete as appropriate.

(N.B. If the assignment is meant to be submitted anonymously, please sign this form and submit it to the Departmental Officer separately from the assignment).

_____ _____

Student Name Signature

_____ _____

Student Name Signature

_____ _____

Student Name Signature

_____ _____

Student Name Signature

_____ _____

Course Code Title of work submitted

_____

Date